JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent.atomic

Class AtomicReferenceArray<E>

java.lang.Object
  java.util.concurrent.atomic.AtomicReferenceArray<E>

Type Parameters:

`E - The base class of elements held in this array`

All Implemented Interfaces:

`Serializable`

```
public class AtomicReferenceArray<E>
extends Object
implements Serializable
```

An array of object references in which elements may be updated atomically. See the `java.util.concurrent.atomic` package specification for description of the properties of atomic variables.

Since:

`1.5`

See Also:

`Serialized Form`

Constructor Summary

Constructors

Constructor	Description
`AtomicReferenceArray(E[] array)`	Creates a new AtomicReferenceArray with the

	same length as, and all elements copied from, the given array.
`AtomicReferenceArray(int length)`	Creates a new AtomicReferenceArray of the given length, with all elements initially null.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`E`	`accumulateAndGet(int i, E x, BinaryOperator<E> accumulatorFunction)`	Atomically updates the element at index `i` with the results of applying the given function to the current and given values, returning the updated value.
`boolean`	`compareAndSet(int i, E expect, E update)`	Atomically sets the element at position `i` to the given updated value if the current value == the expected value.
`E`	`get(int i)`	Gets the current value at position `i`.
`E`	`getAndAccumulate(int i, E x, BinaryOperator<E> accumulatorFunction)`	Atomically updates the element at index `i` with the results of

		applying the given function to the current and given values, returning the previous value.
E	**getAndSet**(int i, **E** newValue)	Atomically sets the element at position i to the given value and returns the old value.
E	**getAndUpdate**(int i, **UnaryOperator**<**E**> updateFunction)	Atomically updates the element at index i with the results of applying the given function, returning the previous value.
void	**lazySet**(int i, **E** newValue)	Eventually sets the element at position i to the given value.
int	**length**()	Returns the length of the array.
void	**set**(int i, **E** newValue)	Sets the element at position i to the given value.
String	**toString**()	Returns the String representation

		of the current values of array.
E	**updateAndGet**(int i, **UnaryOperator**<**E**> updateFunction)	Atomically updates the element at index i with the results of applying the given function, returning the updated value.
boolean	**weakCompareAndSet**(int i, **E** expect, **E** update)	Atomically sets the element at position i to the given updated value if the current value == the expected value.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

AtomicReferenceArray

```
public AtomicReferenceArray(int length)
```

Creates a new AtomicReferenceArray of the given length, with all elements initially null.

Parameters:

`length - the length of the array`

AtomicReferenceArray

```
public AtomicReferenceArray(E[] array)
```

Creates a new AtomicReferenceArray with the same length as, and all elements copied from, the given array.

Parameters:

`array - the array to copy elements from`

Throws:

`NullPointerException - if array is null`

Method Detail

length

```
public final int length()
```

Returns the length of the array.

Returns:

`the length of the array`

get

```
public final E get(int i)
```

Gets the current value at position `i`.

Parameters:

`i - the index`

Returns:

`the current value`

set

```
public final void set(int i,
                      E newValue)
```

Sets the element at position `i` to the given value.

Parameters:

`i - the index`

newValue - the new value

lazySet

```
public final void lazySet(int i,
                          E newValue)
```

Eventually sets the element at position `i` to the given value.

Parameters:

`i - the index`

`newValue - the new value`

Since:

`1.6`

getAndSet

```
public final E getAndSet(int i,
                         E newValue)
```

Atomically sets the element at position `i` to the given value and returns the old value.

Parameters:

`i - the index`

`newValue - the new value`

Returns:

`the previous value`

compareAndSet

```
public final boolean compareAndSet(int i,
                                   E expect,
                                   E update)
```

Atomically sets the element at position `i` to the given updated value if the current value == the expected value.

Parameters:

`i - the index`

`expect - the expected value`

`update - the new value`

Returns:

true if successful. False return indicates that the actual value was not equal to the expected value.

weakCompareAndSet

```
public final boolean weakCompareAndSet(int i,
                                       E expect,
                                       E update)
```

Atomically sets the element at position `i` to the given updated value if the current value == the expected value.

May fail spuriously and does not provide ordering guarantees, so is only rarely an appropriate alternative to `compareAndSet`.

Parameters:

`i` - the index

`expect` - the expected value

`update` - the new value

Returns:

true if successful

getAndUpdate

```
public final E getAndUpdate(int i,
                            UnaryOperator<E> updateFunction)
```

Atomically updates the element at index `i` with the results of applying the given function, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`i` - the index

`updateFunction` - a side-effect-free function

Returns:

the previous value

Since:

1.8

updateAndGet

```
public final E updateAndGet(int i,
                            UnaryOperator<E> updateFunction)
```

Atomically updates the element at index `i` with the results of applying the given function, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`i - the index`

`updateFunction - a side-effect-free function`

Returns:

`the updated value`

Since:

`1.8`

getAndAccumulate

```
public final E getAndAccumulate(int i,
                                E x,
                                BinaryOperator<E> accumulatorFunction)
```

Atomically updates the element at index `i` with the results of applying the given function to the current and given values, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value at index `i` as its first argument, and the given update as the second argument.

Parameters:

`i - the index`

`x - the update value`

`accumulatorFunction - a side-effect-free function of two arguments`

Returns:

`the previous value`

Since:

`1.8`

accumulateAndGet

```
public final E accumulateAndGet(int i,
                                E x,
                                BinaryOperator<E> accumulatorFunction)
```

Atomically updates the element at index `i` with the results of applying the given function to the current and given values, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value at index `i` as its first argument, and the given update as the second argument.

Parameters:

`i - the index`

`x - the update value`

`accumulatorFunction - a side-effect-free function of two arguments`

Returns:

`the updated value`

Since:

`1.8`

toString

```
public String toString()
```

Returns the String representation of the current values of array.

Overrides:

`toString` in class `Object`

Returns:

```
the String representation of the current values of array
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.